

Laboratório 2: Processamento de Bases de Dados em Lote

Danilo Dias Correia Barbosa

Instruções

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

R: Para determinar o percentual global de voos com atraso na chegada superior a 10 minutos (`ARRIVAL_DELAY > 10`) ao processar o arquivo em lotes (*chunks*), é necessário armazenar duas estatísticas suficientes para cada subconjunto (dia, mês e companhia aérea):

- i. **n_atrasos**: A contagem (frequência absoluta) do número de voos com `ARRIVAL_DELAY > 10`.
- ii. **n_total**: O número total de voos válidos (frequência total de observações do grupo).

Justificativa: Não é possível calcular a média direta dos percentuais calculados em cada *chunk*, pois os lotes podem ter quantidades diferentes de voos para um mesmo dia/companhia. Ao manter a soma acumulada de **n_atrasos** e **n_total** ao longo da leitura dos lotes, o percentual final exato é obtido pela razão:

$$\text{Percentual de Atraso} = \frac{\sum n_{atrasos}}{\sum n_{total}}$$

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 - b. Remova observações que tenham valores faltantes em campos de interesse;
 - c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;

- d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;
- e. A função deve receber apenas dois argumentos:
- f. `input`: o conjunto de dados (referente ao lote em questão);
- ii. `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. **É importante** observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library(dplyr)
library(readr)

getStats <- function(input, pos) {
  input %>%
    # 2.1: Filtrar apenas as companhias aéreas solicitadas
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%

    # 2.2: Remover NAs das colunas de interesse
    filter(!is.na(MONTH), !is.na(DAY), !is.na(ARRIVAL_DELAY)) %>%

    # 2.3: Agrupar por dia, mês e cia. aérea
    group_by(MONTH, DAY, AIRLINE) %>%

    # 2.4: Calcular as estatísticas suficientes
    summarise(
      n_atrasos = sum(ARRIVAL_DELAY > 10),
      n_total = n(),
      .groups = "drop"
    )
}
```

- 3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.
- a. Configure o tamanho do lote (chunk) para 100 mil registros;
- b. Configure a função de *callback* para instanciar DataFrames aplicando a função `getStats` criada acima;
- c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```

# Definir tipos das colunas para ler apenas o necessário (economizar memória)
# 'i' = integer, 'c' = character, '_' = skip (ignorar coluna)
col_spec <- cols_only(
  MONTH = col_integer(),
  DAY = col_integer(),
  AIRLINE = col_character(),
  ARRIVAL_DELAY = col_double()
)

# Leitura em lotes de 100.000 linhas
stats_parciais <- read_csv_chunked(
  file = "flights.csv.zip",
  callback = DataFrameCallback$new(getStats),
  chunk_size = 100000,
  col_types = col_spec
)

```

4. Crie uma função chamada `computeStats` que:
 - a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
 - b. Retorne as informações em um `tibble` contendo apenas as seguintes colunas: i. Cia: sigla da companhia aérea; ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`); iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

```

computeStats <- function(dados_acumulados) {
  dados_acumulados %>%
    # Reagrupar por mês, dia e companhia aérea
    group_by(MONTH, DAY, AIRLINE) %>%

    # Somar as estatísticas suficientes de todos os chunks
    summarise(
      total_atrasos = sum(n_atrasos),
      total_voos = sum(n_total),
      .groups = "drop"
    ) %>%

    # Criar a estrutura final exigida
    transmute(
      Cia = AIRLINE,

```

```

    Data = as.Date(paste(2015, MONTH, DAY, sep = "-")),
    Perc = total_atrasos / total_voos
  )
}

# Aplicar a função para obter a base final
stats_finais <- computeStats(stats_parciais)

```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.
 - a. Instale e carregue os pacotes `ggcal` e `ggplot2`.
 - b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
 - c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá: i. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`. ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`. iii. Retornar para o usuário a base do calendário criada acima.
 - d. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `CAA`, `CDL`, `cUA` e `cUS`.
 - e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Titulo")`.

```

library(ggplot2)
library(dplyr)
library(lubridate)

# Paleta definida na Questão 5.2
pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")

# Função alternativa sem usar o pacote ggcal
baseCalendario <- function(stats, cia) {
  df_cia <- stats %>%
    filter(Cia == cia) %>%
    mutate(
      SemanaMes = (mday(Data) - 1) %/% 7 + 1, # Semana dentro do mês
    )
}

```

```

    DiaSemana = wday(Data, label = TRUE, week_start = 1), # Seg a Dom
    Mes = month(Data, label = TRUE, abbr = TRUE)
  )

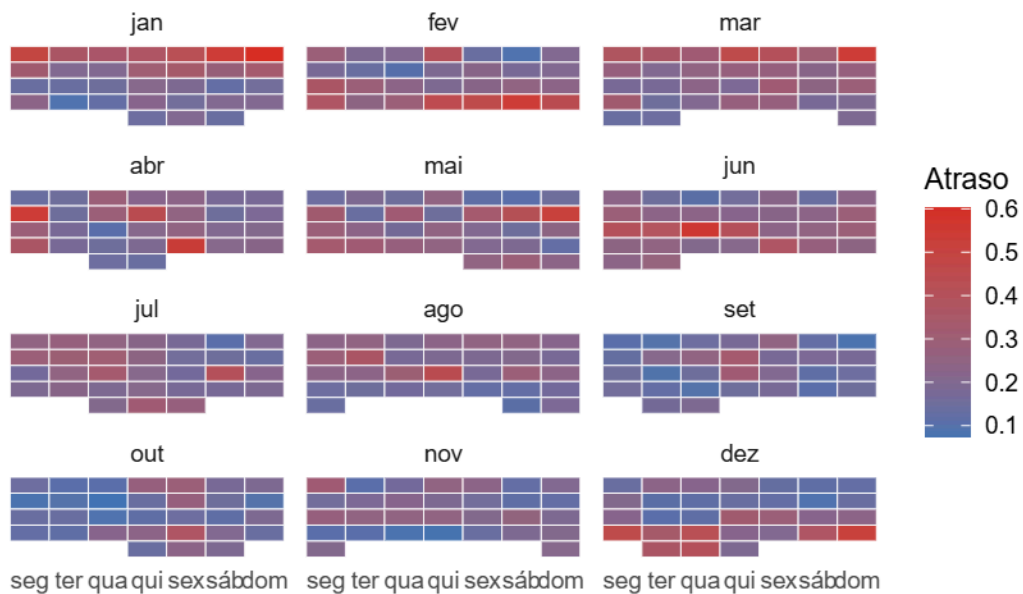
ggplot(df_cia, aes(x = DiaSemana, y = factor(SemanaMes, levels = 5:1), fill = Perc)) +
  geom_tile(color = "white") +
  facet_wrap(~ Mes, ncol = 3) +
  labs(x = NULL, y = NULL, fill = "Atraso") +
  theme_minimal() +
  theme(
    axis.text.y = element_blank(),
    axis.ticks = element_blank(),
    panel.grid = element_blank()
  )
}

# 5.4: Gerar os calendários
cAA <- baseCalendario(stats_finais, "AA")
cDL <- baseCalendario(stats_finais, "DL")
cUA <- baseCalendario(stats_finais, "UA")
cUS <- baseCalendario(stats_finais, "US")

# 5.5: Exibir os mapas com a paleta e os títulos
cAA + pal + ggtitle("Percentual de Atrasos em 2015 - American Airlines (AA)")

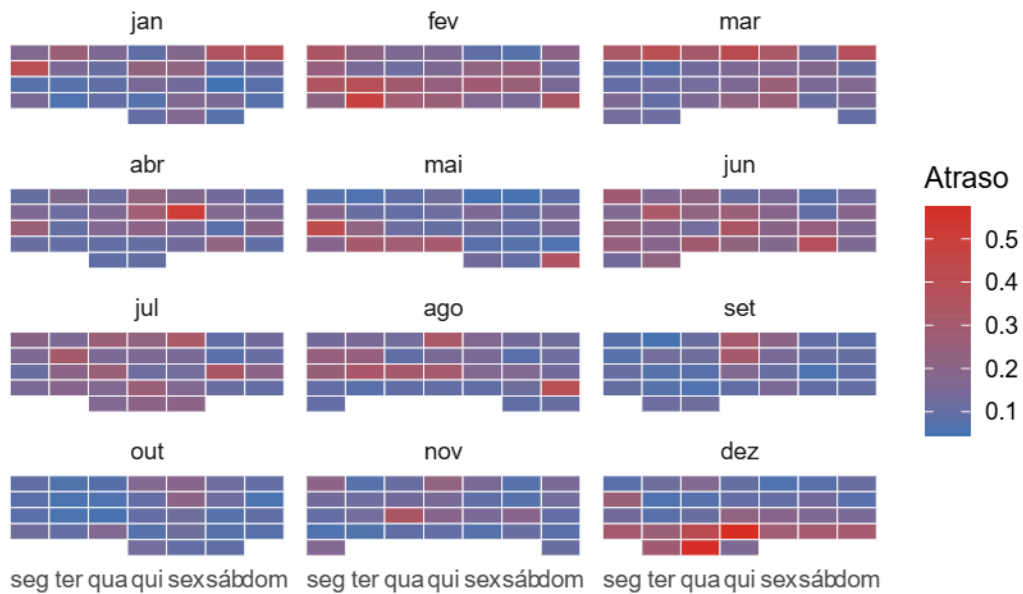
```

Percentual de Atrasos em 2015 – American Airlines (AA)



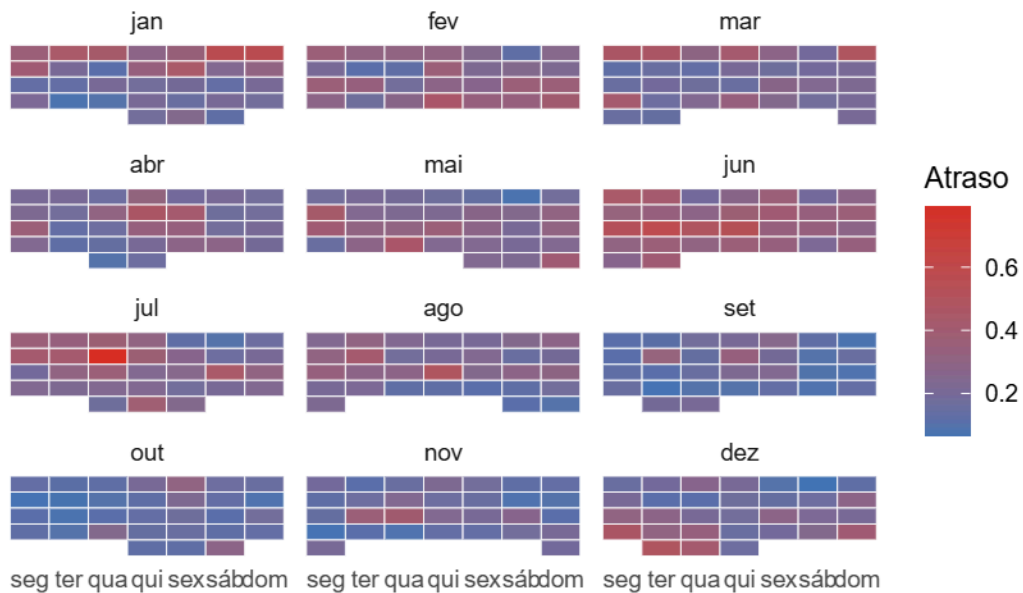
```
cDL + pal + ggtitle("Percentual de Atrasos em 2015 – Delta Air Lines (DL)")
```

Percentual de Atrasos em 2015 – Delta Air Lines (DL)



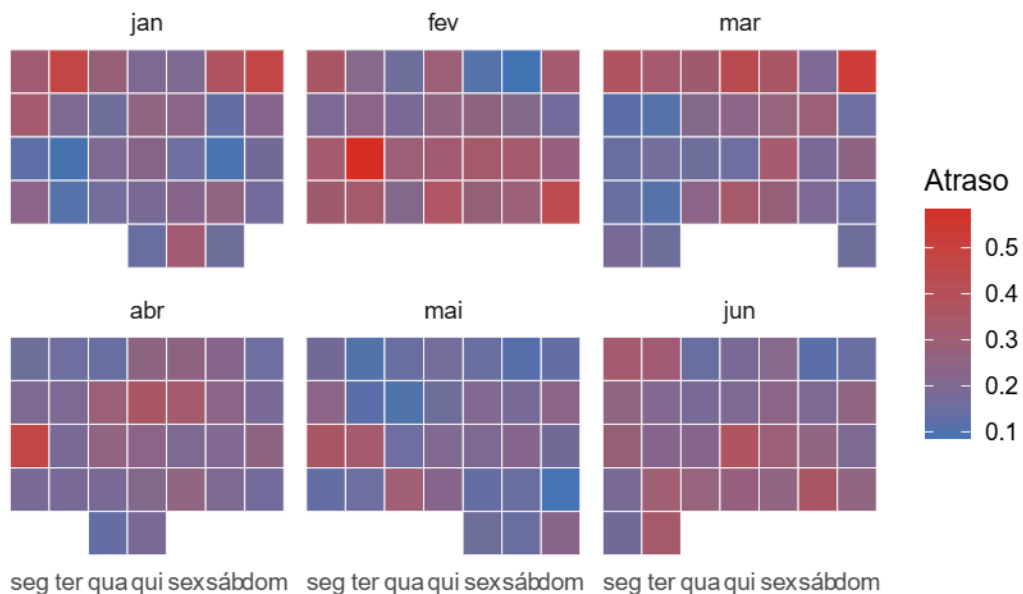
```
cUA + pal + ggtitle("Percentual de Atrasos em 2015 - United Airlines (UA)")
```

Percentual de Atrasos em 2015 – United Airlines (UA)



```
cUS + pal + ggtitle("Percentual de Atrasos em 2015 - US Airways (US)")
```

Percentual de Atrasos em 2015 – US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-24 21:41:22 -03"
```